

STATIC AND DYNAMIC ARRAY

1. Concept Overview

Feature	Static Array	Dynamic Array
Size	Fixed at compile time	Flexible (can be adjusted at runtime)
Memory Allocation	Done at <i>compile time</i> (stack memory)	Done at <i>runtime</i> (heap memory)
Control	Managed automatically by compiler	You control allocation (new) and deallocation (delete)
Performance	Faster (no heap overhead)	Slightly slower (heap operations take time)
Flexibility	Cannot resize after creation	Can resize by creating new arrays and copying

2. Static Array in C++

Definition

A static array is an array whose size is known at compile time and cannot change during execution.

cpp

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    // Static array declaration with fixed size
```

```
    int arr[5] = {10, 20, 30, 40, 50};
```

```
    cout << "Static Array Elements: ";
```

```
    for (int i = 0; i < 5; i++) {
```

```
        cout << arr[i] << " ";
```

```

}

cout << endl;

// Trying to access out of bounds -> undefined behavior
// arr[5] = 60; // NOT ALLOWED (beyond the defined size)

return 0;
}

```

Key Points

- Stored in stack memory.
- Size must be constant (like 5 or a const variable).
- Destroyed automatically when the function ends.
- Very efficient but inflexible.

3. Dynamic Array in C++

Definition

A dynamic array is allocated in heap memory using the new operator. It can be created and resized *during program execution*.

cpp

```

#include <iostream>

using namespace std;

```

```

int main() {

    int size;

    cout << "Enter array size: ";

    cin >> size;
}

```

```

// Dynamic array allocation
int* dynArr = new int[size];

// Input elements
cout << "Enter " << size << " elements: ";
for (int i = 0; i < size; i++) {
    cin >> dynArr[i];
}

// Output
cout << "Dynamic Array Elements: ";
for (int i = 0; i < size; i++) {
    cout << dynArr[i] << " ";
}
cout << endl;

// Free the allocated memory
delete[] dynArr; // Always free after usage
dynArr = nullptr; // Avoid dangling pointer

return 0;
}

```

Key Points

- Stored in heap memory.
- Size determined at runtime.

- Must be manually deleted to prevent memory leaks.
 - Use new and delete[] carefully.
-

4. Dynamic Array Resizing (Advanced Concept)

You cannot directly “resize” a C++ array.

However, you can simulate resizing like this:

cpp

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int size = 3;
```

```
    int* arr = new int[size] {1, 2, 3};
```

```
    cout << "Original Array: ";
```

```
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
```

```
    // Need a bigger array
```

```
    int newSize = 5;
```

```
    int* newArr = new int[newSize];
```

```
    // Copy elements
```

```
    for (int i = 0; i < size; i++) {
```

```
        newArr[i] = arr[i];
```

```
    }
```

```

// Add new elements
newArr[3] = 4;
newArr[4] = 5;

// Delete old array
delete[] arr;
arr = newArr;
size = newSize;

cout << "\nResized Array: ";
for (int i = 0; i < size; i++) cout << arr[i] << " ";
cout << endl;

delete[] arr;
return 0;
}

```

This is how C++ STL `std::vector` dynamically manages memory internally!

5. When to Use Which

Use Case	Recommended Type
Size is constant	Static array
Size depends on user input	Dynamic array
Frequent resizing needed	Use <code>std::vector</code> (dynamic internally)

6. Summary

Feature	Static	Dynamic
Memory	Stack	Heap
Size	Fixed	Flexible
Allocation	Compile time	Runtime
Resizing	Not possible	Possible with manual copy
Managed by	Compiler	Programmer
Safer Alternative	–	std::vector